

Mastering Tree Traversal



Logic, Visualization, & Code

BASED ON CHAI AUR CODE | BINARY TREE SERIES

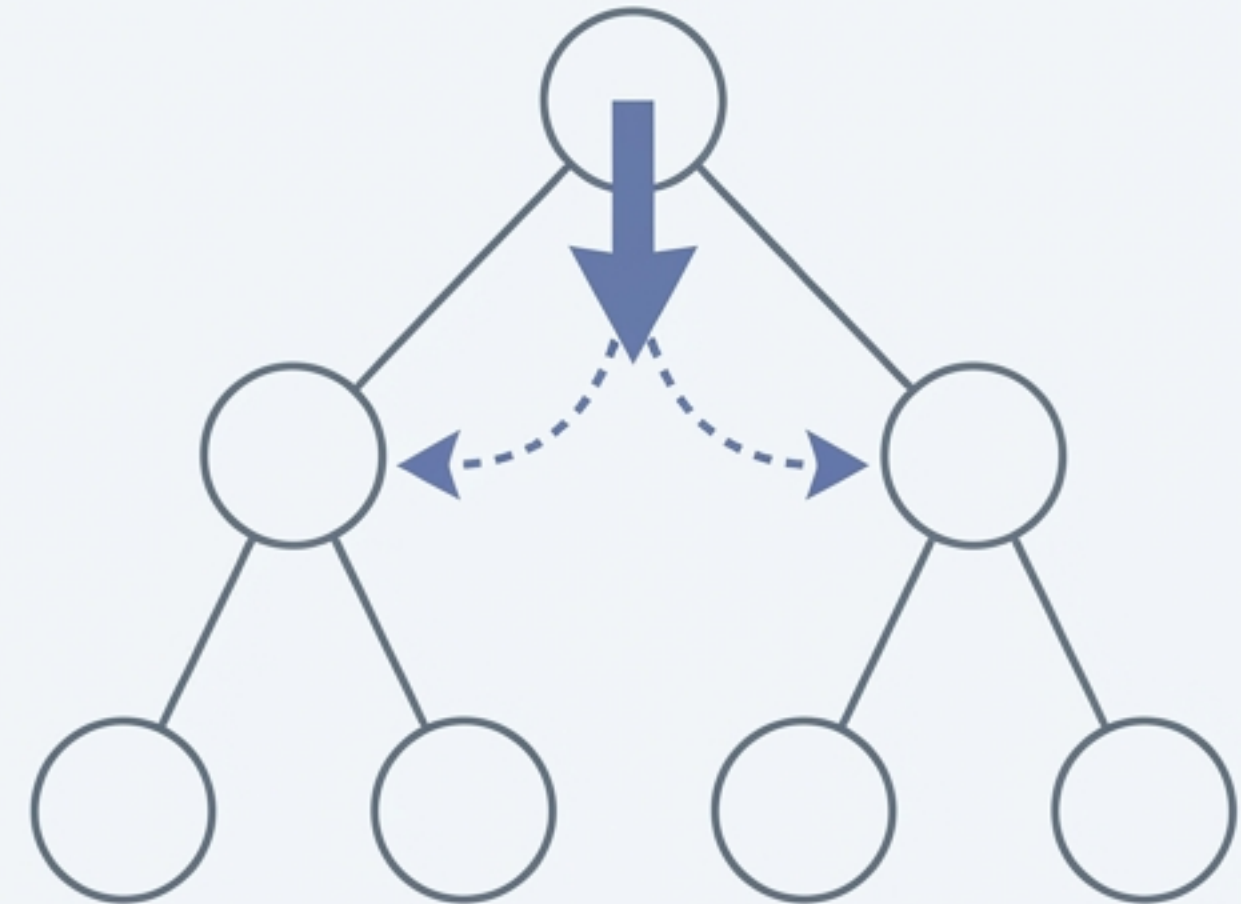
The Traversal Dilemma

Linear Data



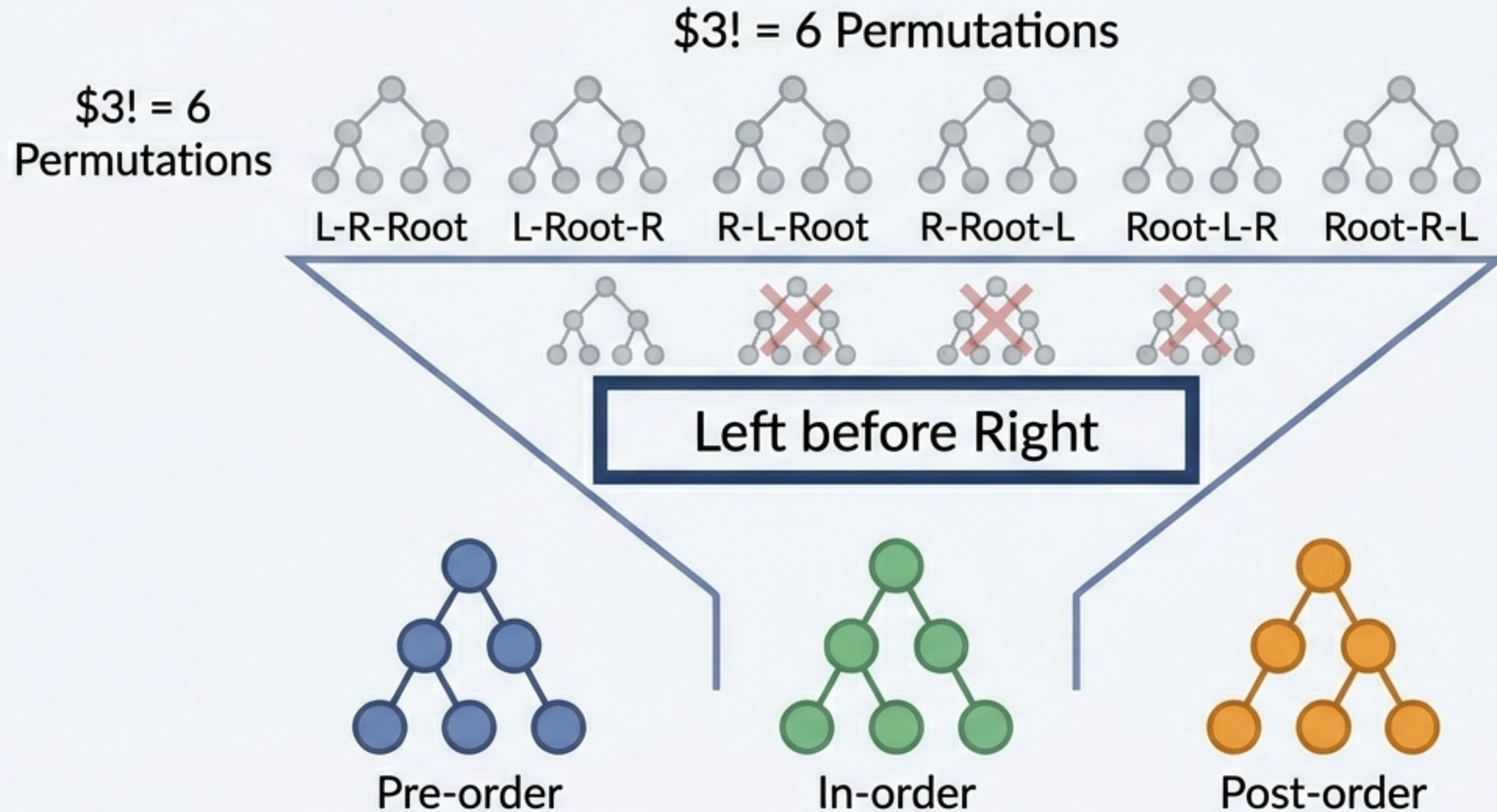
One clear path. $i++$

Hierarchical Data



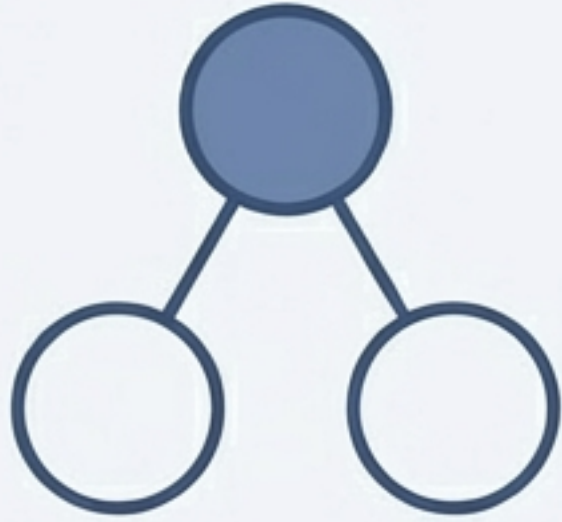
Non-Linear. Choice required
at every node.

Reducing the Possibilities



The Golden Rule: **Left always comes before Right.** The only variable is the **Root**.

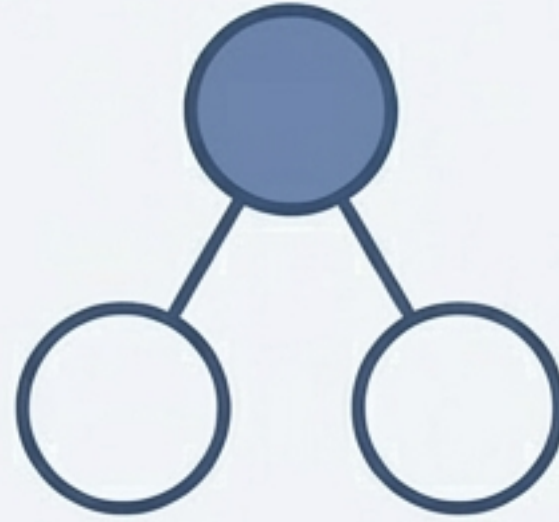
The “Root Position” Mnemonic



Pre-order

Root is **First**

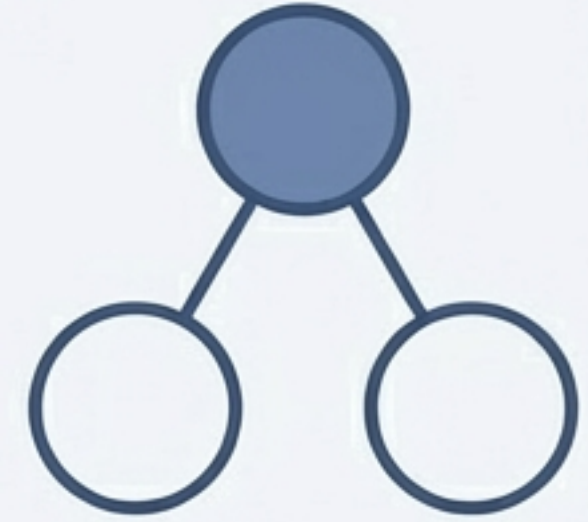
Root → Left → Right



In-order

Root is **In** the Middle

Left → Root → Right



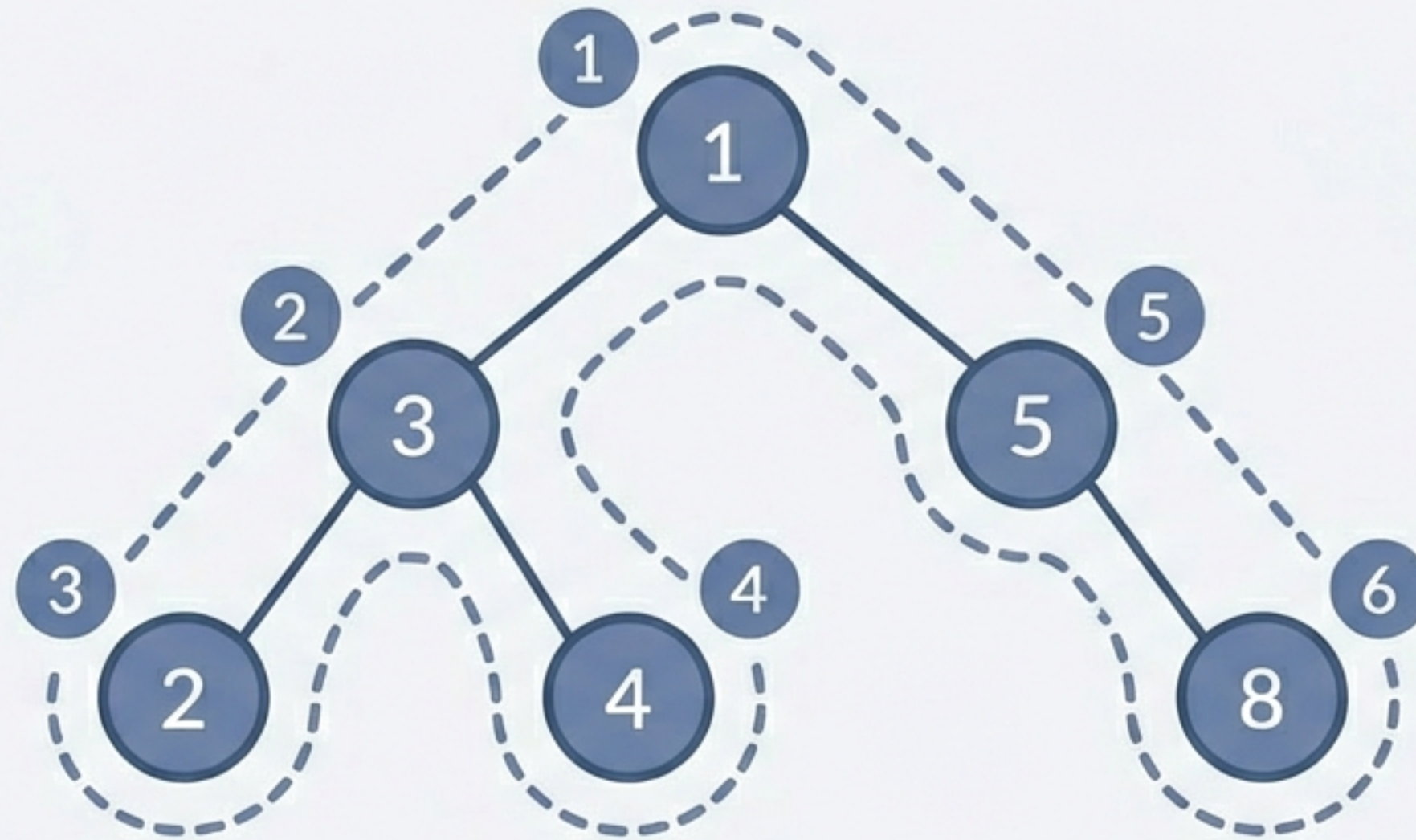
Post-order

Root is **Last**

Left → Right → Root

Visualizing Pre-order

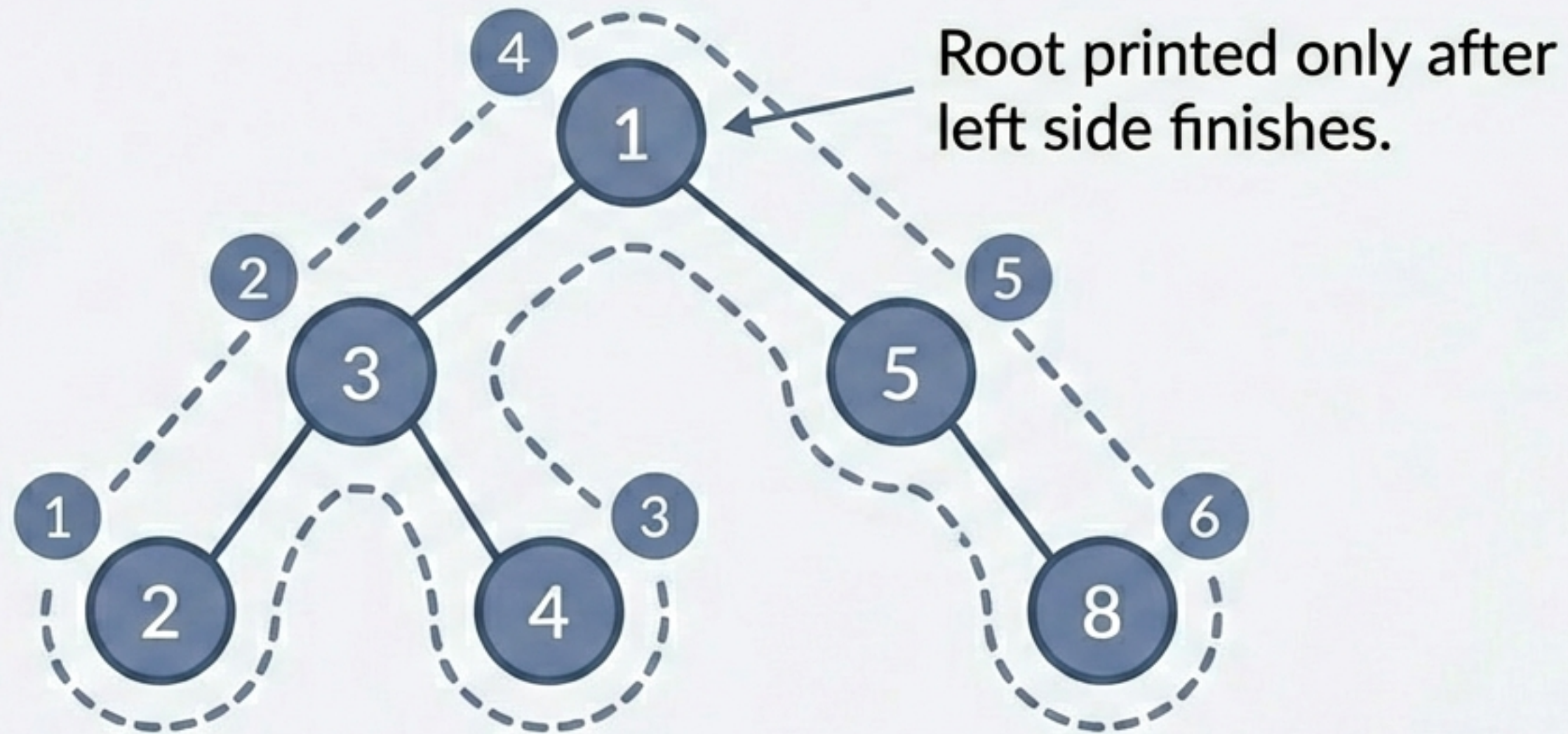
Visit Root → Descend Left → Descend Right



Output: 1 3 2 4 5 8

Visualizing In-order

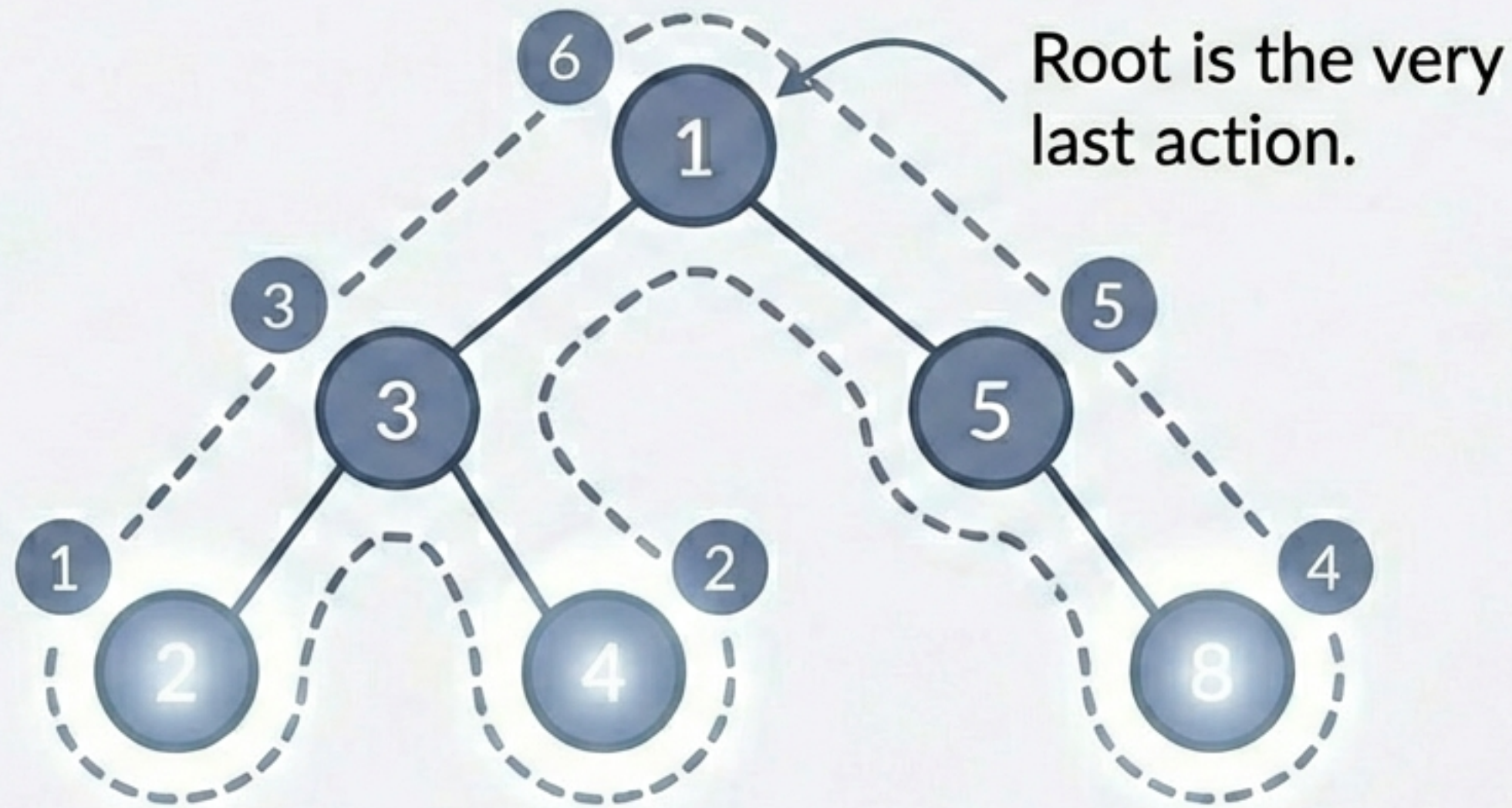
Deep Left → Root → Right



Output: 2 3 4 1 5 8

Visualizing Post-order

Children First, Parents Last



Output: 2 4 3 8 5 1

The Python Structure: Node Class

```
class Node:
    def __init__(self, val):
        self.left = None
        self.right = None
        self.data = val

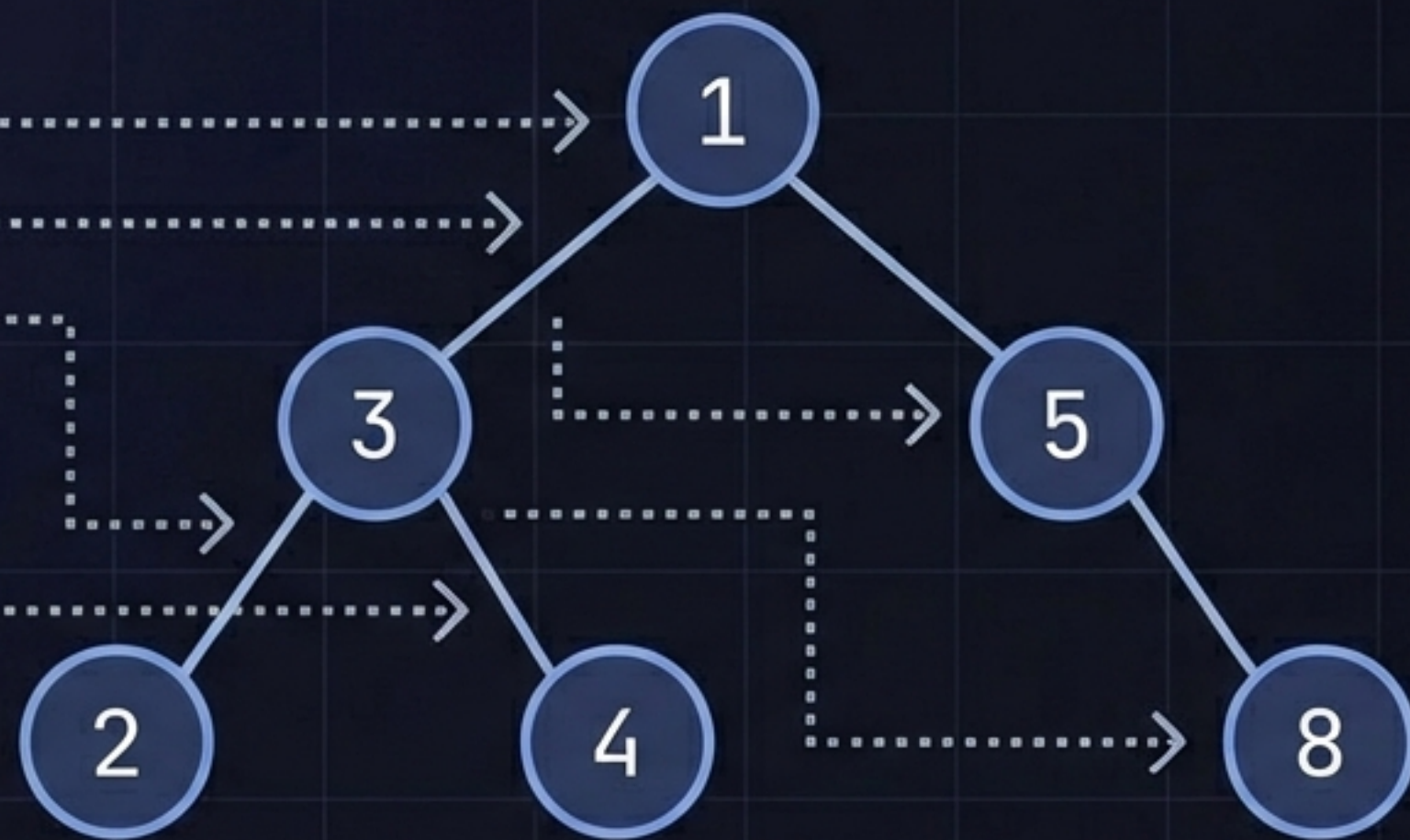
# Directional Pointers, not Prev/Next.
```



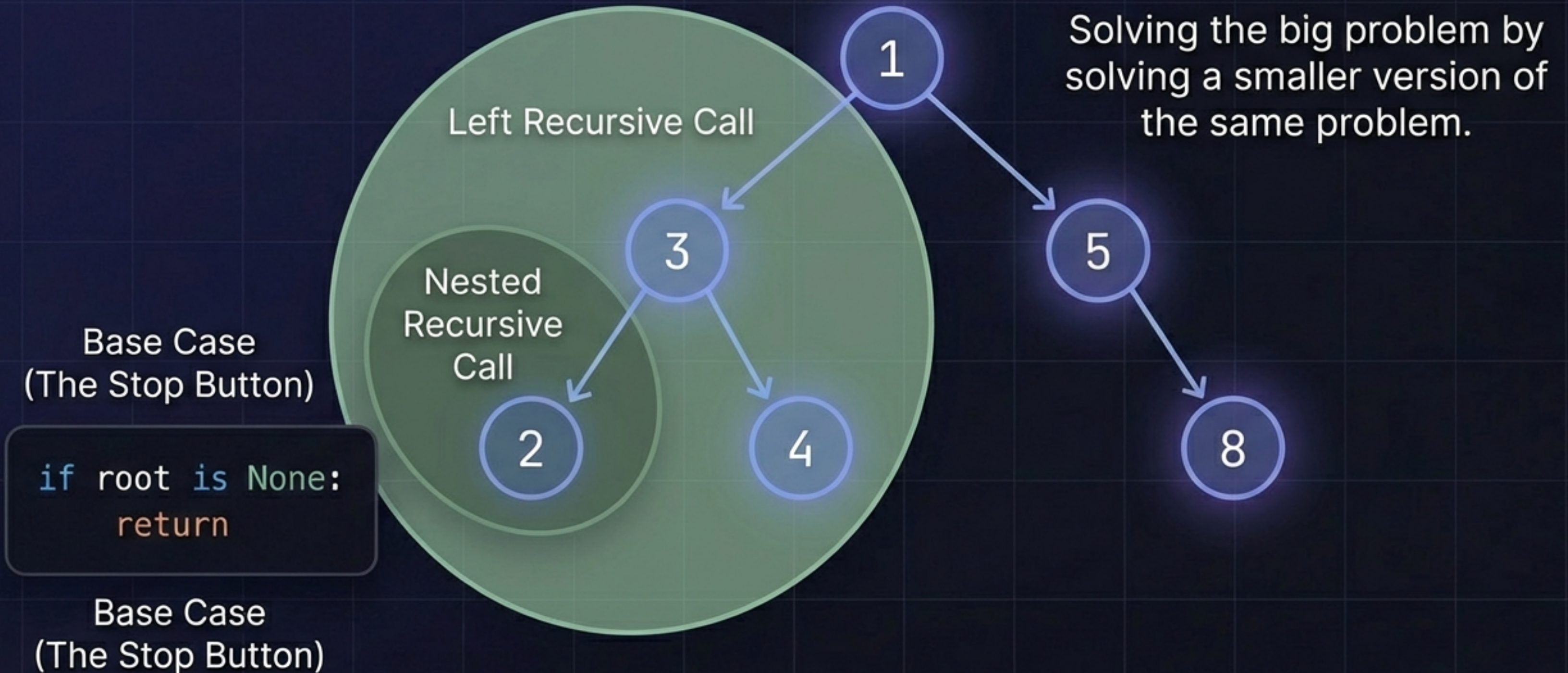
Building the Test Tree

```
root = Node(1)
root.left = Node(3)
root.right = Node(5)

# Linking grandchildren
root.left.left = Node(2)
root.left.right = Node(4)
root.right.right = Node(8)
```



The Anatomy of Recursion



The Pre-order Algorithm

```
def preorder(root):  
    if root is None: return  
  
    print(root.data)           # 1. Process  
    preorder(root.left)       # 2. Recurse Left  
    preorder(root.right)      # 3. Recurse Right
```



Action happens
before recursion

The Power of One Line

Pre-order

```
print(root.data)  
recurse(left)  
recurse(right)
```

In-order

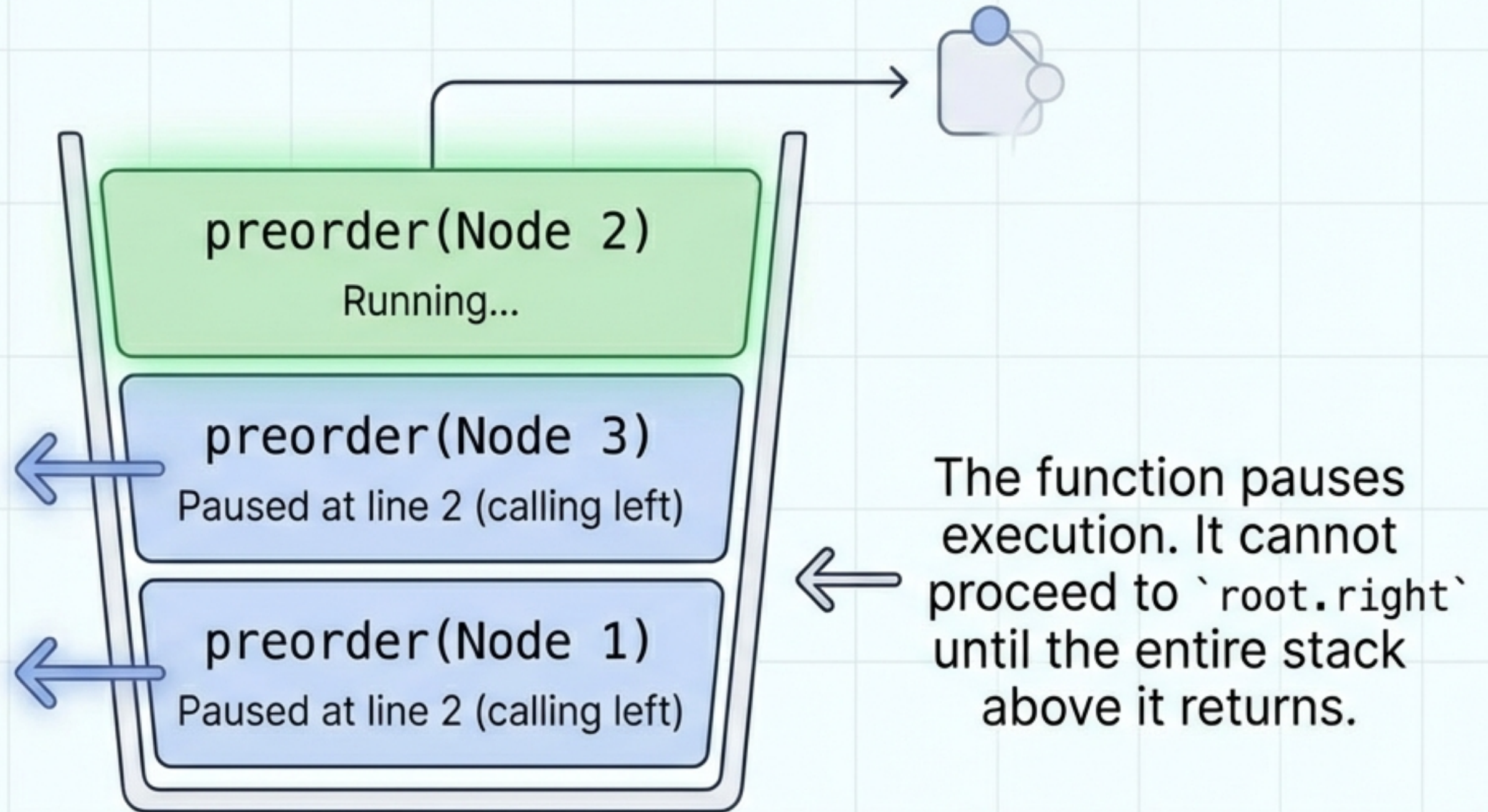
```
recurse(left)  
print(root.data)  
recurse(right)
```

Post-order

```
recurse(left)  
recurse(right)  
print(root.data)
```

Logic is dictated by the position of the print statement.

Under the Hood: Tracing the Stack

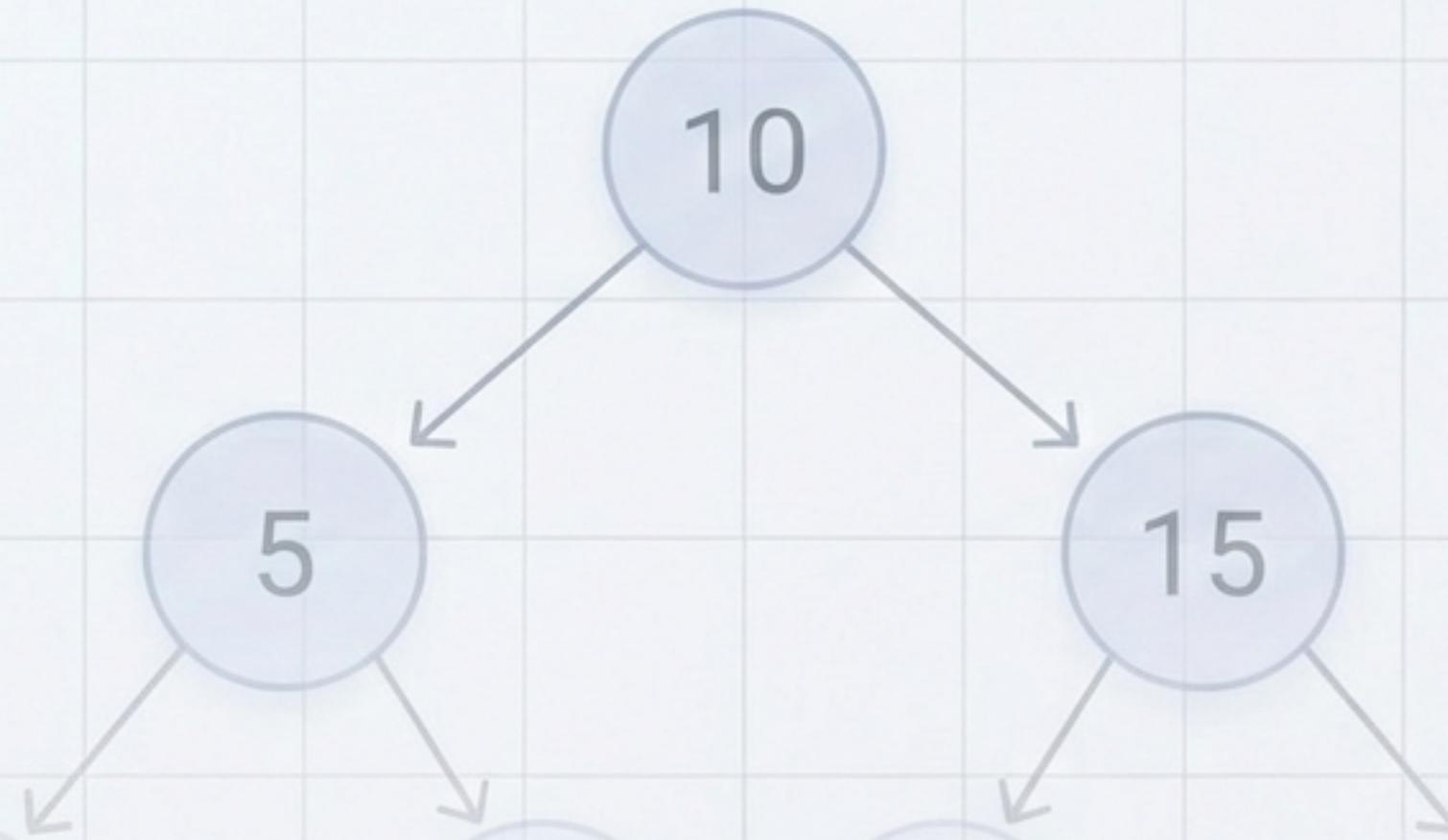


Summary & Key Takeaways

Traversal	Rule	Code Position	Output Pattern
Pre-order	Root First	Print before recursion	1 3 2...
In-order	Root Middle	Print between recursions	2 3 4...
Post-order	Root Last	Print after recursions	2 4 3...

Tip: Always draw the tree and trace the path manually first.

What's Next?



We know how to move through a tree. Now we need to organize it.

Coming Up: Binary Search Trees (BST)